



Kafka Integration Guide - Event Management System



Table des Matières

1. [Vue d'ensemble](#)

2. [Architecture Kafka](#)

3. [Topics Kafka](#)

4. [Services Producers](#)

5. [Services Consumers](#)

6. [Configuration](#)

7. [Démarrage](#)

8. [Tests](#)

9. [Monitoring](#)

10. [Troubleshooting](#)



Vue d'ensemble

Pourquoi Kafka ?

L'intégration de **Apache Kafka** dans notre architecture microservices permet de :

- ☒ **Découpler les services** - Communication asynchrone sans dépendances directes

☒ **Améliorer les performances** - Pas de blocage lors des opérations lentes (emails, notifications)

☒ **Garantir la fiabilité** - Les messages sont persistés et peuvent être rejoués

☒ **Faciliter l'évolution** - Ajouter de nouveaux consommateurs sans modifier les producteurs

☒ **Créer un audit trail** - Historique complet de tous les événements métier

Cas d'usage implémentés

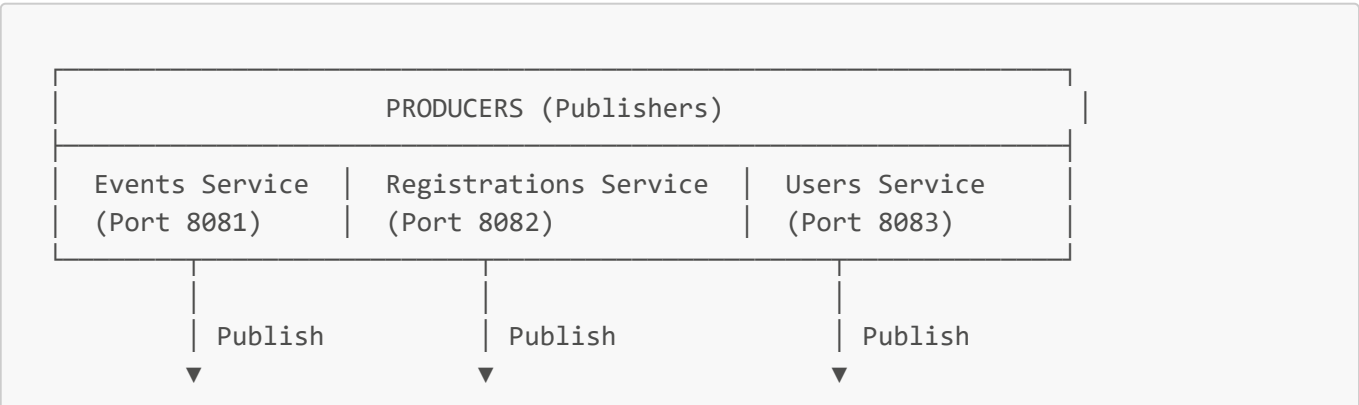
1. **Notifications asynchrones** - Envoi d'emails sans bloquer les opérations principales

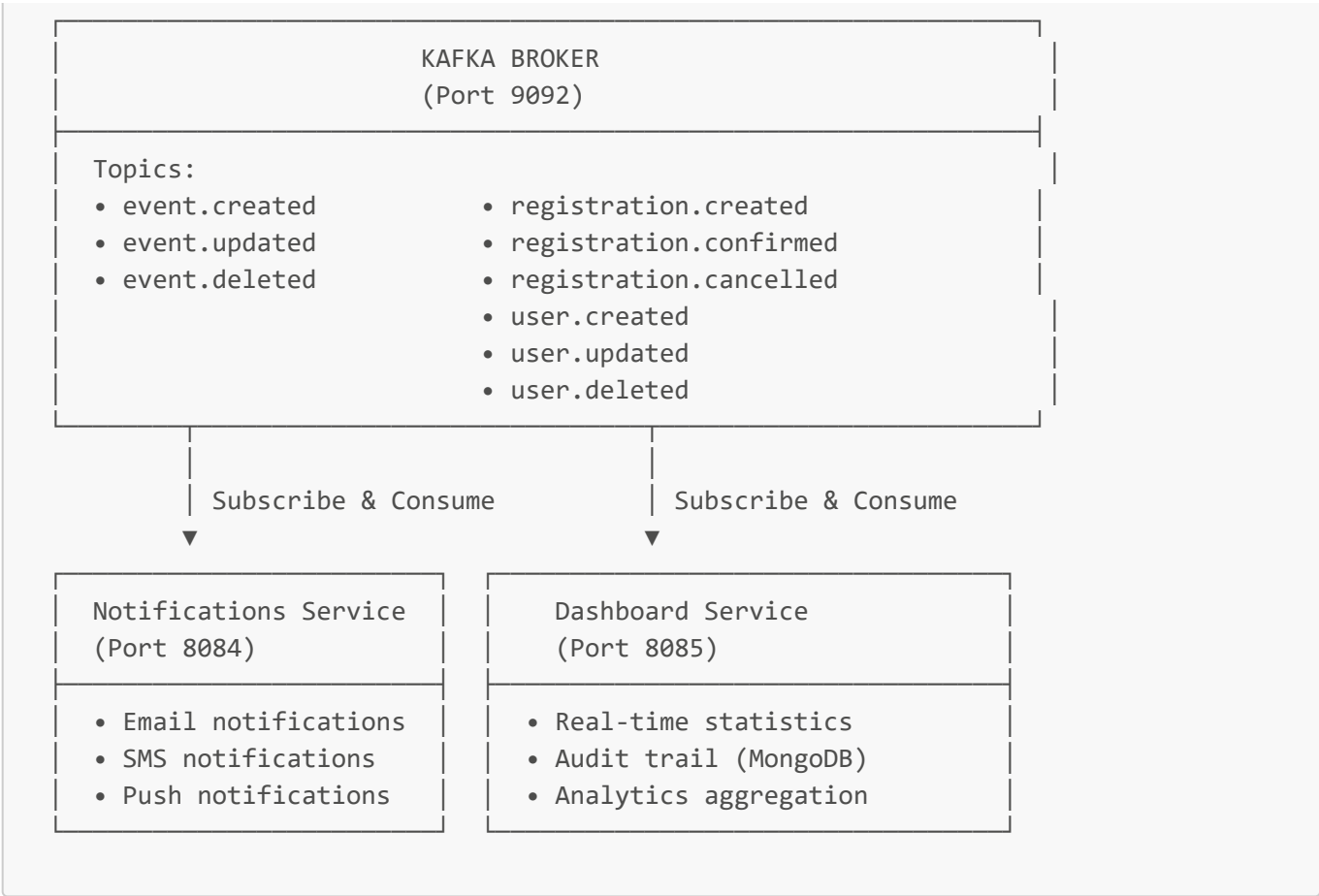
2. **Dashboard temps réel** - Agrégation des statistiques en temps réel

3. **Audit et compliance** - Historisation complète de tous les événements



Architecture Kafka





Composants

- **Zookeeper** (Port 2181) - Coordination et gestion du cluster Kafka
- **Kafka Broker** (Port 9092) - Serveur de messagerie
- **Kafka UI** (Port 8090) - Interface web de monitoring

📧 Topics Kafka

Events Topics

Topic	Producer	Consumers	Description
event.created	events-service	notifications, dashboard	Événement créé
event.updated	events-service	dashboard	Événement mis à jour
event.deleted	events-service	dashboard	Événement supprimé

Registrations Topics

Topic	Producer	Consumers	Description
registration.created	registrations-service	notifications, dashboard	Inscription créée
registration.confirmed	registrations-service	notifications, dashboard	Inscription confirmée
registration.cancelled	registrations-service	dashboard	Inscription annulée

Users Topics

Topic	Producer	Consumers	Description
<code>user.created</code>	users-service	notifications, dashboard	Utilisateur créé (welcome email)
<code>user.updated</code>	users-service	dashboard	Utilisateur mis à jour
<code>user.deleted</code>	users-service	dashboard	Utilisateur supprimé

Services Producers

Events Service

Fichiers clés :

- `kafka/EventMessage.java` - DTO du message
- `kafka/EventPublisher.java` - Service de publication
- `service/EventService.java` - Appelle le publisher

Exemple de publication :

```
@ApplicationScoped
public class EventService {
    @Inject
    EventPublisher eventPublisher;

    public Event create(Event event) {
        repository.persist(event);

        // Publication asynchrone vers Kafka
        eventPublisher.publishEventCreated(event);

        return event;
    }
}
```

Registrations Service

Fichiers clés :

- `kafka/RegistrationMessage.java`
- `kafka/RegistrationPublisher.java`
- `service/RegistrationService.java`

Users Service

Fichiers clés :

- `kafka/UserMessage.java`

- kafka/UserPublisher.java
- service/UserProfileService.java

Services Consumers

Notifications Service

Consommation des messages :

```
@ApplicationScoped
public class NotificationConsumer {

    @Incoming("event-created")
    public void onEventCreated(EventMessage message) {
        // Envoyer notification email à l'organisateur
        LOG.infof("Sending notification for event: %s", message.getTitle());
    }

    @Incoming("registration-created")
    public void onRegistrationCreated(RegistrationMessage message) {
        // Envoyer email de confirmation au participant
        LOG.infof("Sending confirmation email");
    }

    @Incoming("user-created")
    public void onUserCreated(UserMessage message) {
        // Envoyer email de bienvenue
        LOG.infof("Sending welcome email to: %s", message.getEmail());
    }
}
```

Dashboard Service

Le Dashboard Service a **deux consommateurs** :

1. DashboardConsumer (Statistiques temps réel)

```
@ApplicationScoped
public class DashboardConsumer {
    private final AtomicInteger totalEvents = new AtomicInteger(0);

    @Incoming("event-created")
    public void onEventCreated(EventMessage message) {
        totalEvents.incrementAndGet();
        // Mettre à jour les stats en base
    }
}
```

2. AuditConsumer (Historisation complète)

```
@ApplicationScoped
public class AuditConsumer {
    @Inject
    AuditLogService auditLogService;

    @Incoming("event-created")
    public void auditEventCreated(EventMessage message) {
        auditLogService.createAuditLog(
            "EVENT", "CREATED", message.getEventId(),
            message, "event.created", message.getTimestamp()
        );
    }
}
```

Modèle Audit :

```
@MongoEntity(collection = "audit_logs")
public class AuditLog extends PanacheMongoEntity {
    private String eventType; // EVENT, REGISTRATION, USER
    private String action; // CREATED, UPDATED, DELETED
    private String entityId; // ID de l'entité
    private String payload; // Message complet en JSON
    private String topic; // Topic Kafka
    private Instant timestamp;
}
```

Configuration

application.properties (Producer - Events Service)

```
# Kafka Bootstrap
kafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}

# Outgoing channels
mp.messaging.outgoing.event-created.connector=smallrye-kafka
mp.messaging.outgoing.event-created.topic=event.created
mp.messaging.outgoing.event-created.value.serializer=io.quarkus.kafka.client.serialization.ObjectMapperSerializer

mp.messaging.outgoing.event-updated.connector=smallrye-kafka
mp.messaging.outgoing.event-updated.topic=event.updated
mp.messaging.outgoing.event-updated.value.serializer=io.quarkus.kafka.client.serialization.ObjectMapperSerializer
```

```
mp.messaging.outgoing.event-deleted.connector=smallrye-kafka
mp.messaging.outgoing.event-deleted.topic=event.deleted
mp.messaging.outgoing.event-deleted.value.serializer=io.quarkus.kafka.client.serialization.ObjectMapperSerializer
```

application.properties (Consumer - Notifications Service)

```
kafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}

# Incoming channels
mp.messaging.incoming.event-created.connector=smallrye-kafka
mp.messaging.incoming.event-created.topic=event.created
mp.messaging.incoming.event-created.value.deserializer=io.quarkus.kafka.client.serialization.ObjectMapperDeserializer
mp.messaging.incoming.event-created.specific.type=com.eventmgmt.notifications.dto.EventMessage
mp.messaging.incoming.event-created.group.id=notifications-service-events
```

docker-compose.yml

```
zookeeper:
  image: confluentinc/cp-zookeeper:7.6.0
  ports:
    - "2181:2181"
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181

kafka:
  image: confluentinc/cp-kafka:7.6.0
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  depends_on:
    - zookeeper

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  ports:
    - "8090:8080"
  environment:
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:29092
```

Démarrage

1. Démarrer l'infrastructure

```
# Démarrer Kafka, Zookeeper et Kafka UI
docker-compose up -d zookeeper kafka kafka-ui

# Vérifier que Kafka est prêt
docker logs kafka
```

2. Démarrer les services

```
# 1. Infrastructure services
cd services/eureka-server
mvn spring-boot:run

cd services/gateway-service
mvn spring-boot:run

# 2. Producer services
cd services/events-service
mvn quarkus:dev

cd services/registrations-service
mvn quarkus:dev

cd services/users-service
mvn quarkus:dev

# 3. Consumer services
cd services/notifications-service
mvn quarkus:dev

cd services/dashboard-service
mvn quarkus:dev
```

3. Accéder à Kafka UI

```
http://localhost:8090
```

Vous pouvez voir :

- Les topics créés

- Les messages dans chaque topic
- Les consumer groups
- Les partitions

Tests

Test 1 : Créer un événement

```
# Via Gateway
POST http://localhost:8080/api/events
Content-Type: application/json

{
  "title": "Test Kafka Event",
  "location": "Paris",
  "startAt": "2026-05-01T10:00:00",
  "endAt": "2026-05-01T18:00:00",
  "organizerId": "org123",
  "category": "CONFERENCE"
}
```

Vérifications :

1. ☒ Événement créé dans events-service
2. ☒ Message publié dans topic `event.created`
3. ☒ Notifications Service log la réception
4. ☒ Dashboard Service met à jour les stats
5. ☒ AuditLog créé dans MongoDB

Test 2 : Créer une inscription

```
POST http://localhost:8080/api/registrations
Content-Type: application/json

{
  "eventId": "event-id-here",
  "participantId": "user123"
}
```

Vérifications :

1. ☒ Message dans topic `registration.created`
2. ☒ Email de confirmation (log dans notifications-service)
3. ☒ Stats mises à jour dans dashboard

Test 3 : Créer un utilisateur


```
POST http://localhost:8080/api/users
Content-Type: application/json

{
  "username": "testuser",
  "email": "test@example.com",
  "firstName": "Test",
  "lastName": "User",
  "password": "Password123!"
}
```

Vérifications :

1. ☒ Message dans topic `user.created`
2. ☒ Welcome email (log)
3. ☒ Audit log créé

Monitoring

Kafka UI (<http://localhost:8090>)

Fonctionnalités :

-  Liste des topics
-  Messages dans chaque topic
-  Consumer groups et leur lag
-  Métriques de performance

Logs des services

```
# Voir les logs Kafka dans Events Service
docker logs events-service | grep "Published"

# Voir les logs dans Notifications Service
docker logs notifications-service | grep "Received"

# Voir les logs dans Dashboard Service
docker logs dashboard-service | grep "Dashboard"
```

Vérifier les Consumer Groups

```
# Dans le container Kafka
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --
list

# Détails d'un consumer group
```

```
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 \
--describe --group notifications-service-events
```

Troubleshooting

Problème : Les messages ne sont pas consommés

Solution :

```
# 1. Vérifier que Kafka est démarré
docker ps | grep kafka

# 2. Vérifier les topics
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --list

# 3. Vérifier les consumer groups
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --list

# 4. Vérifier les logs du service
docker logs notifications-service
```

Problème : Erreur de sérialisation

Vérifier que :

- Les DTOs (EventMessage, RegistrationMessage, UserMessage) sont identiques entre producer et consumer
- Lombok est bien configuré (@Data, @NoArgsConstructor, @AllArgsConstructor)
- ObjectMapperSerializer/Deserializer est configuré

Problème : Kafka ne démarre pas

```
# Vérifier Zookeeper d'abord
docker logs zookeeper

# Nettoyer et redémarrer
docker-compose down
docker-compose up -d zookeeper
# Attendre 10 secondes
docker-compose up -d kafka
```

Problème : Consumer lag élevé

Causes possibles :

- Consumer trop lent (traitement lourd)
- Pas assez de partitions
- Problèmes réseau

Solution :

```
# Augmenter le nombre de partitions
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --alter --topic event.created --partitions 3
```

Ressources

Documentation officielle

- [Apache Kafka](#)
- [Quarkus Kafka](#)
- [SmallRye Reactive Messaging](#)

Commandes utiles

```
# Lister les topics
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --list

# Créer un topic manuellement
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --create --topic test-topic --partitions 3 --replication-factor 1

# Lire les messages d'un topic
docker exec -it kafka kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic event.created --from-beginning

# Décrire un topic
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --describe --topic event.created
```

☒ Checklist de validation

- ☒ Zookeeper démarré (port 2181)
- ☒ Kafka démarré (port 9092)
- ☒ Kafka UI accessible (port 8090)
- ☒ Events Service publie vers Kafka
- ☒ Registrations Service publie vers Kafka
- ☒ Users Service publie vers Kafka
- ☒ Notifications Service consomme les messages
- ☒ Dashboard Service agrège les stats

- ☒ AuditLogs enregistrés dans MongoDB
 - ☒ Tous les topics sont créés automatiquement
 - ☒ Les consumer groups fonctionnent
 - ☒ Pas de lag important dans les consumers
-

Prochaines étapes (Améliorations futures)

1. **Partitionnement avancé** - Distribuer les messages par organizerId ou eventId
 2. **Dead Letter Queue** - Gérer les messages en erreur
 3. **Schema Registry** - Valider les schémas des messages avec Avro
 4. **Kafka Streams** - Agrégations complexes en temps réel
 5. **Monitoring avancé** - Prometheus + Grafana
 6. **Multi-DC replication** - Haute disponibilité
-

Date de dernière mise à jour : 13 avril 2026

Version Kafka : 7.6.0

Version Quarkus : 3.8.4